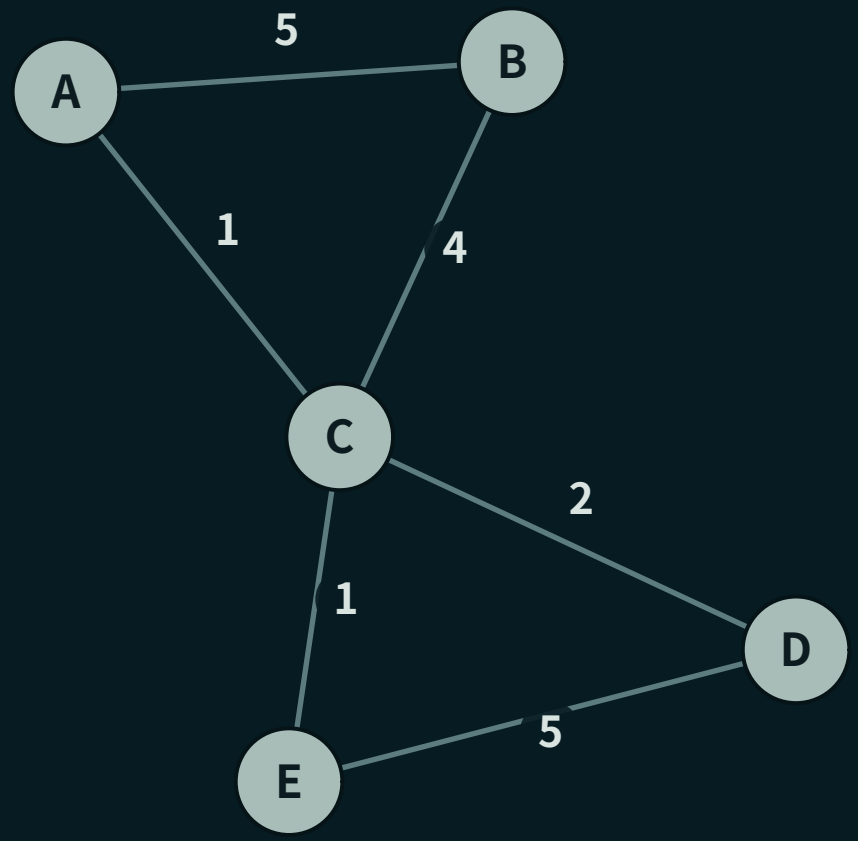
A number for every point

Spectral bisection · the Laplacian, the Fiedler vector and the sweep

Learning Clustering · original author xingguangyaoyu (Issue #5)

The problem

- Both groups non-empty; left and right not distinguished
- 5 nodes: $(2^5 - 2) \div 2 = 15$ splits (not 16)
- Balance window: 40%–60% per group (here 2–3 a side)
- Output: the split, cut, Fiedler order, sweep table, numerical diagnostics
- The spectral method gives a good candidate, not a guaranteed optimum



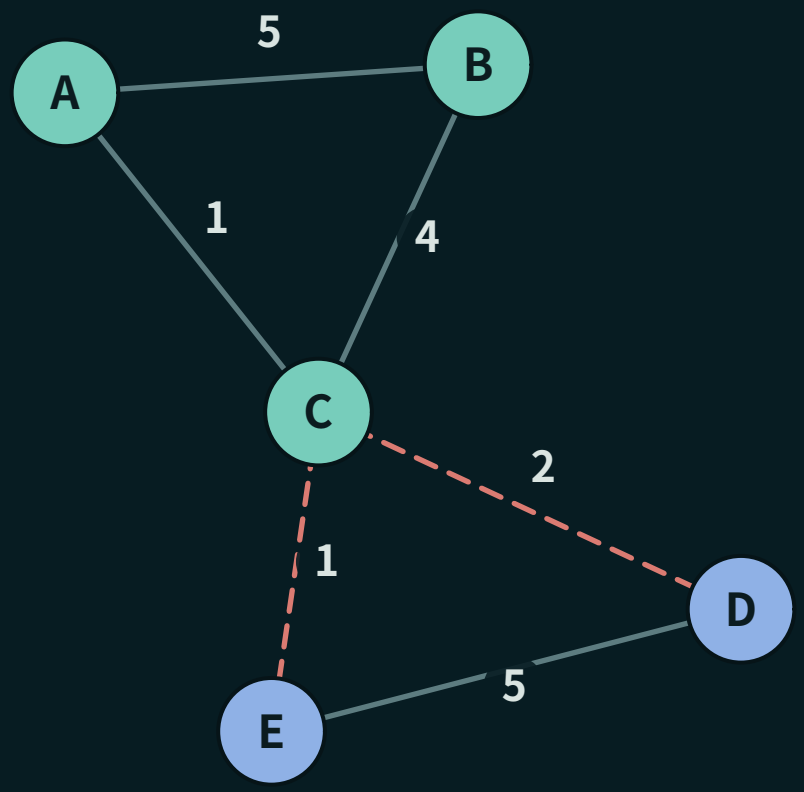
The Laplacian L = D − A

- Diagonal: the degree (sum of incident edge weights)
- Off the diagonal: minus the edge weight; 0 where there is no edge
- Each edge contributes to four entries: ±w
- Self-check: L symmetric, every row sums to 0
- Note: this D is the degree matrix, not the inter-cluster delay of Module 09

	A	B	C	D	E	
A	6	−5	−1	0	0	row sum 0
B	−5	9	−4	0	0	row sum 0
C	−1	−4	8	−2	−1	row sum 0
D	0	0	−2	7	−5	row sum 0
E	0	0	−1	−5	6	row sum 0

The quadratic form: why tightly linked points get close numbers

- $x^T L x = \Sigma\, w(u,v) \cdot (x_u - x_v)^2$
- With $x = \pm 1$: a cut edge contributes $4w \rightarrow x^T L x = 4 \times$ cut
- Relax: x any real vector, orthogonal to all-ones
- The relaxed optimum: the eigenvector of the second-smallest eigenvalue
- Springs are only an analogy; this page runs no physical simulation



teal = +1 (A, B, C) · blue = −1 (D, E)

$x^T L x = 4 \times$ cut = 12

The Fiedler vector and its conditions

- $\lambda 1 = 0$ (the all-ones vector); $\lambda 2 \approx 1.7733$
- Fiedler vector: A 0.519, B 0.418, C 0.106, D −0.489, E −0.553
- Conditions: the graph is connected; $\lambda 2$ is not degenerate
- Sign flip: $-x$ is an eigenvector too, and only swaps sides

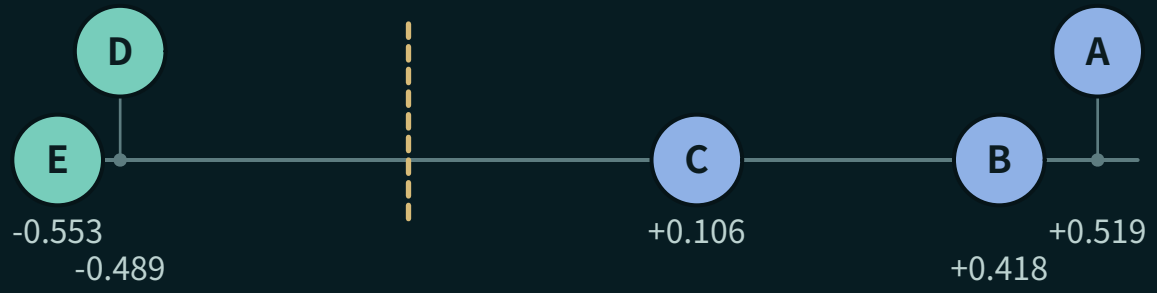


$\lambda = 0, 1.7733, 8.4551, 11.6053, 14.1663$

Fiedler order: E, D, C, B, A

The sweep

- Order: E, D, C, B, A
- Cuts for k = 1⋯4: 6, 3, 5, 6
- The 40/60 window allows only k = 2, 3
- Take k = 2: DE | ABC, cut 3
- Cutting "right in the middle", k = 3, gives 5



k	Left	cut	40/60 window
1	E	6	—
2	ED	3	legal
3	EDC	5	legal
4	EDCB	6	—

Two misunderstandings, corrected

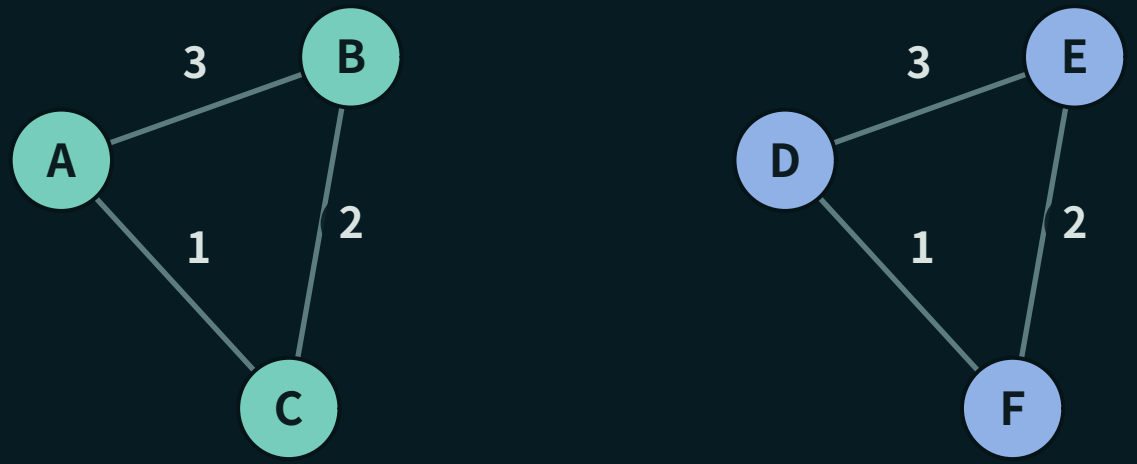
- "With no constraint the optimum isolates one node": wrong; the least single-node cut is 6, ABC | DE is only 3
- Say instead: with no constraint a group can come out too small
- "The stored vector is a solver": wrong; its residual 0.0044 is rounding
- The stored values are not recomputed when the graph changes; this page solves live, residual about 10^{-15}

Isolate	cut = degree
A	6
B	9
C	8
D	7
E	6

Vector source	Residual
solved live	3.8e-15
original, 3 decimals	0.0044

A disconnected graph

- Two components → $\lambda_1 = \lambda_2 = 0$
- The Fiedler vector only shows who shares a component
- Check connectivity and degeneracy before using it
- The lab's diagnostics panel gives a warning



$\lambda_1 = \lambda_2 = 0$: the order only reflects components

Pseudocode and numerical self-checks

- Line 1: build L, check symmetry and row sums
- Lines 2–3: eigendecomposition; check connectivity, degeneracy, residual
- Lines 4–5: fix the sign, then sort
- Lines 6–7: sweep n − 1 positions, take the least cut inside the window

```
1  L ← D − A (check symmetry, row sums 0)
2  (λ, x) ← eigendecomposition, ascending
3  check connectivity, degeneracy, residual
4  f ← eigenvector of λ2 (fix the sign)
5  order ← sort by f, ascending
6  for k = 1 ⋯ n−1: compute cut
7  return the k with least cut in the window
```

Limits and uses

- Gives a good initial bisection, often handed to KL/FM to improve
- Unnormalised L; normalised variants give different orders
- At scale, use sparse iterative solvers (such as Lanczos)
- Springs are only an analogy

Summary

L = D − A; the Fiedler vector gives every point a number.

Sort, sweep the n − 1 cut points, choose only inside the balance window; a sign flip only swaps sides.

Next module: start from this bisection and keep improving it with pairwise swaps.